

# 应用层服务语义

把字节组织成名字、资源与操作

408 · NET-08

## 1 本册定位

本 Topic 回答：端到端传输只提供 byte stream 或 datagram，应用怎样定义对象、名字、操作、响应和状态，使不同程序能够共同完成 DNS、Web、文件与邮件服务？

本册拥有 application architecture、DNS、HTTP semantics、FTP 与电子邮件的服务分工，以及 application protocol 对 transport service 的选择逻辑。

DHCP 虽以 UDP application protocol 形式运行，但在本项目中由 IP 配置与转发拥有，因为其主要输出是主机进入 IP 世界所需的启动配置。

## 2 独立阅读词典：应用协议在定义什么

| 术语                        | 本册中的可操作定义                                                           | 不要混成什么                         |
|---------------------------|---------------------------------------------------------------------|--------------------------------|
| application protocol      | 约定对象、名字、消息字段、动作顺序和响应含义                                              | 不只是一个端口号                       |
| client/server 与 P2P       | 前者由 client 请求、server 提供；后者参与者可同时请求和提供                               | 都不等于 TCP/UDP                   |
| resource / representation | resource 是 URI 标识的抽象对象；representation 是某时刻某格式的可传输表示                 | response body 不是 resource 全部身份 |
| URI / URL                 | URI 是资源标识语法；URL 是带定位信息的常见 URI 形式                                    | 不等于 DNS 返回的 IP                 |
| domain / zone             | domain 是名字树名称；zone 是权威管理者实际负责的数据边界                                  | 不等于物理网络区域                      |
| resolver / authoritative  | resolver 代客户端查询并缓存；authoritative server 对自己的 zone 给权威答案；TTL 是缓存时间预算 | resolver 不是权威数据 Owner          |
| request/response 与邮件/文件协议 | request 表达意图，response 返回状态/表示；SMTP 传邮件，POP3/IMAP 访问邮箱，FTP 分离控制与数据   | 都不是 TCP ACK                    |

本册所说的“语义”就是一个具体问题的答案：客户端发出这个消息后，服务器应把哪个对象当作目标、执行什么动作、返回什么状态，以及下一步是否允许继续。这样读 HTTP、DNS、邮件和 FTP 时，

不会把字段格式误当成服务含义。

### 3 根本问题：transport 不知道“这段 byte 意味着什么”

TCP 可以保证 byte 顺序，却不知道这段 byte 是域名查询、HTTP request、邮件命令还是文件内容。Application protocol 必须定义：

Application object → Name/identifier → Operation  
→ Message representation → Response/state transition

它不仅规定报文格式，还规定动作的语义与交互顺序。相同 transport 可以承载完全不同服务；相同服务语义也可以演进出不同 transport bindings。

## 4 应用架构：谁长期持有状态

### 4.1 Client/Server

server 通常在稳定地址/名字上等待请求并持有共享服务状态；client 主动发起交互。集中服务便于管理、一致性和发现，代价是容量、故障和运营中心化。

### 4.2 Peer-to-Peer

peer 同时消费和提供资源，容量可随参与者增加，但发现、可用性、信任、NAT 穿越和一致性更复杂。

C/S 与 P2P 是状态与职责的组织方式，不等同于 TCP/UDP，也不由“是否有服务器进程”一个表象决定。

## 5 DNS：把一个集中表拆成层次化授权系统

### 5.1 为什么单一 hosts 文件失败

全网共享一份名字表会遇到：更新冲突、分发延迟、单点容量与管理边界。DNS 的生成链是：

Global flat file fails → Hierarchical namespace → Delegated zones  
→ Distributed authoritative servers → Resolver + cache

Domain namespace 是名字树；zone 是某个 authority 实际管理的数据边界。树的结构与管理分区相关，但 domain 不等于一台主机或一个物理网络。

### 5.2 四类对象

- stub resolver：应用/OS 发起查询的本地接口；
- recursive resolver：代用户完成后续查询并缓存；
- authoritative name server：对某 zone 的数据负责；
- root/TLD/other delegated servers：通过 referral 把查询逐步引向 authority。

### 5.3 一次 cache miss 的解析轨迹

```

1 application asks local resolver
2 -> recursive resolver checks cache
3 -> root referral identifies TLD servers
4 -> TLD referral identifies authoritative servers
5 -> authoritative answer returns records
6 -> resolver caches answer/referrals under TTL
7 -> result returns to application

```

迭代查询返回“我不知道最终答案，但下一步问谁”；递归查询把继续查找的责任交给被询问方。实际 client-to-recursive 常请求递归，resolver 对 authority 链执行迭代式追踪。

### 5.4 Cache 的正确性边界

cache 用旧信息换低延迟和低查询负载。TTL 限制可复用时间，但 DNS 不是瞬时全局一致系统：更新在 TTL 生命周期内可能与缓存旧值并存。

DNS 并非只能用 UDP。经典普通查询常用 UDP；响应截断、zone transfer 等情形可使用 TCP，现代部署还存在加密传输。408 做题要服从题设，不能把“DNS=UDP”写成协议语义。

## 6 HTTP：操作资源的统一语义

### 6.1 WWW：分布式超文档系统

WWW 不是某一个 protocol 或“所有 Internet 服务”的同义词。它至少包含：browser/client、Web server、由 URL/URI 标识的 resource、HTML 等 document/representation，以及把文档与其他资源连接起来的 hyperlink。HTTP 定义 client 与 server 操作这些资源的请求/响应语义；DNS 可把 URL 中的 host name 解析为 IP；transport 再承载 HTTP message。

URL identifies resource → DNS locates host → HTTP requests representation → HTML links reveal dependencies

“网页”常是一个主文档与图片、样式、脚本等多个对象组成的依赖图，不是一条 HTTP response 必然完整返回的单文件。URL 提供标识与定位语法，HTML 提供超文本表示，HTTP 提供操作/传输语义；三者不能互相替代。

HTTP 的核心对象不是“网页文件”，而是由 URI 标识的 resource。消息传输的是 resource 的 representation，method 表达 client 希望对目标执行的语义。

Target resource → Method semantics → Request metadata/content → Status → Representation/metadata

### 6.2 Resource 不等于 representation

同一 resource 可根据时间、语言、编码或内容协商产生不同 representation。response body 是某次选定表示，不等于资源本身的全部身份。

### 6.3 Method 属性改变重试决策

- safe: 语义上只读, 例如 GET/HEAD;
- idempotent: 重复执行与执行一次具有相同预期 effect, 例如 PUT、DELETE 以及 safe methods;
- POST 通常不假定 idempotent。

这些属性不是礼貌标签, 而是 cache、proxy、client 在失败后能否自动重试的重要决策依据。

### 6.4 Stateless 不等于“服务没有状态”

HTTP stateless 表示每个 request 的语义不依赖 server 必须记住此前 protocol request context。应用仍可通过 cookie、token、database 和 server-side session 建立业务状态。

### 6.5 语义与连接版本分离

- HTTP/1.1 和 HTTP/2 通常在 TCP 上承载;
- HTTP/3 使用 QUIC over UDP;
- 核心 method、status、representation 和 cache semantics 在版本间保持共同定义。

因此“HTTP 使用 TCP”是特定版本/部署的 transport mapping, 不是 HTTP resource semantics 本身。

### 6.6 Transport endpoint 不等于 HTTP authority: 同一 IP:port 可以承载多个名字空间

TCP 的四元组只把字节流交给某个 server endpoint; HTTP 仍需要知道当前请求属于哪个应用名字。对 HTTP/1.1, Host 请求首部携带目标 host/authority; HTTP/2/3 使用等价的 :authority 语义。因此两个不同域名即使 DNS 最终解析到同一 server IP、并使用相同 TCP port, 也可以由同一服务进程按 application authority 选择不同 virtual host / resource namespace。

Domain/Authority  $\neq$  IP address  $\neq$  TCP endpoint

这里不能反向推断: 共享 IP/port 不表示两个域名是同一应用资源; 不同域名也不要求必须分配不同 IP 或不同 port。题目若问“同一服务器怎样区分两个基于名称的站点”, 应在 HTTP message semantics 中找 Host/authority, 而不是强行修改传输层标识。

## 7 Web 性能从依赖图生成

取得一个 URL 可能依次触发: DNS、transport/security establishment、HTTP request/response、HTML parsing、更多 subresource requests。

总时间不能只数“几个 RTT”, 必须先画依赖关系: 哪些步骤串行、哪些 connection 可复用、哪些 object 可并行、哪些命中 cache。

Persistent connection 减少重复建立连接; multiplexing 允许多个 request/response 共享连接并发推进; cache 用 freshness/validation 状态减少传输。三者解决的瓶颈不同。

## 8 FTP：控制状态与数据传输分离

FTP 的问题是跨异构系统可靠操作和传输文件。它使用持久 control connection 交换命令与会话状态，并为目录/文件数据建立独立 data connection。

这种 out-of-band control 让命令不被大文件数据阻塞，也带来额外连接管理、主动/被动模式和 NAT/firewall 适配成本。FTP 的两条 TCP connections 是应用层设计，不是 TCP 自动生成的“控制通道”。

## 9 电子邮件：提交、转发、存储、读取不是一个协议

邮件系统至少有：user agent、mail server、message transfer 和 mailbox access。

```
1 sender user agent
2 -> submission / SMTP transfer
3 -> one or more mail servers
4 -> recipient mailbox
5 -> POP3 or IMAP access
6 -> recipient user agent
```

- SMTP 负责提交/服务器间 push 式传送；
- POP3 提供较简单的 mailbox download/access 模型；
- IMAP 更强调服务器端 mailbox 状态与多设备同步；
- MIME 扩展 message representation，使非 ASCII 内容和附件可被邮件体系承载。

“发邮件”和“收邮件”必须分开建模，不能把所有过程归给 SMTP。

## 10 应用模型核验层：C/S 与 P2P 的可计算边界

C/S 与 P2P 首先是角色、状态与资源位置的模型，不是 TCP/UDP 的同义词。若服务器以  $u_s$  上传一个大小为  $F$  的文件给  $N$  个客户，客户最低下载速率为  $d_{min}$ ，在忽略核心拥塞的带宽下界模型中：

$$D_{C/S} = \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{min}} \right\}.$$

服务器上行要产出  $N$  份完整副本，所以第一项随  $N$  线性增长。

若对等方  $i$  也能以上传速率  $u_i$  分享文件，则

$$D_{P2P} = \max \left\{ \frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum_i u_i} \right\}.$$

第三项的分子、分母同时随参与者增加；这叫 self-scalability，但不等于“人越多单个下载越快”。若所有 peer 都只下载、 $u_i = 0$ ，P2P 的供给项退化为 C/S 的服务器瓶颈。文件分块让新 peer 尚未下载完整文件时就能开始上传，是自扩展性真正落地的条件。

资源定位还要独立建模：集中目录（如集中 tracker）可以只集中“谁拥有”，文件仍由 peer 直接传；全分布式洪泛降低单点但增加查询开销；分块/邻居交换让定位与传输都分散。看到“P2P”不能直接推断没有任何中心辅助节点。

## 11 DNS 核验层：名字树、授权区与记录类型

一个 label 最多 63 个字符，完整域名文本总长通常不超过 255 个字符，大小写不区分；这些是名字语法边界，不是 IP 地址或子网掩码。domain 是名字树中的节点/子树，zone 是某个 authoritative server 实际拥有的管理数据边界，二者不能互换。

| 记录    | 指向/用途              | 题面判据               |
|-------|--------------------|--------------------|
| A     | 主机名到 IPv4 地址       | 返回 32-bit address  |
| AAAA  | 主机名到 IPv6 地址       | 返回 128-bit address |
| NS    | zone 的权威服务器        | referral/委派链的下一责任者 |
| CNAME | 别名到 canonical name | 还需继续解析目标名          |
| MX    | 域的邮件交换服务器及优先级      | SMTP relay 选择下一跳   |
| PTR   | 反向命名（地址到名称）        | reverse lookup     |

root server、TLD server、authoritative server 与 recursive resolver 的 Owner 不同。root/TLD 通常返回 referral (NS 与必要 glue)，authoritative server 才对自己 zone 内最终 RR 负责；recursive resolver 代表 stub 承担继续查询、缓存和错误返回。迭代查询的响应把“下一步问谁”交还给 caller，递归查询则把后续责任交给被询问方。

一次 cache miss 的可审计顺序是：stub 发送 name/type/class/transaction ID；resolver 查本地 cache；若未命中，依次询问 root、TLD、authoritative；校验 response identity 与 RR；按 positive 或 negative TTL 缓存；向 stub 返回 answer 或错误。缓存提高性能却产生旧值窗口，TTL 到期是可复用期限，不是全网同时更新的保证。NXDOMAIN/negative answer 也可按其缓存语义保留，不能把“没查到”写成永远不存在。

普通 DNS query 常走 UDP 53；响应截断、zone transfer 或题设要求可靠字节流时可使用 TCP 53。现代加密 DNS 是 transport deployment 变化，不改变 hierarchy、authority 与 RR 语义。看到 ‘DNS=UDP’ 只能说明常见承载，不能替代上述流程。

## 12 HTTP 核验层：报文语法、方法属性与连接代价

HTTP/1.x 请求/响应的语义骨架可以写成：

```
request line:  method + target + version,
               headers + blank line + [optional body],
response:     version + status code/reason,
               headers + blank line + [representation].
```

常见方法的核心不是英文动词而是重试与缓存属性：GET/HEAD 通常 safe；PUT/DELETE 语义上 idempotent；POST 通常用于创建/触发处理且不能默认安全重试。状态码可按类别审计：1xx informational、



2xx success、3xx redirection/cache validation、4xx client request、5xx server failure；状态码不是 TCP ACK，也不代表业务最终完成。

HTTP stateless 只说明每个 request 的协议语义不要求 server 保存上一次 request context；cookie、token、session/database 仍可维护业务状态。resource 是被 URI 标识的对象，representation 是某次按语言、编码或时间协商出的可传输表示；缓存控制必须针对 representation freshness/validation，而不能把 body 当永久资源身份。

性能题先分连接代价与消息代价。非持久 HTTP/1.0 的简单模型中，DNS 命中且无安全握手时，建立 TCP 约 1 RTT、发送 request 并得到首字节再约 1 RTT，之后再加对象传输时间；持久连接复用后可省后续建连 RTT；HTTP/1.1 pipeline、HTTP/2 multiplexing 与 HTTP/3/QUIC 改变并发与传输映射，但不改变 resource/method/status 语义。具体 RTT 公式必须按题设是否串行、缓存是否命中、对象是否并行来画依赖 DAG，不能套一个固定“网页 = 2 RTT”。

代理/Web cache 可能在 client 与 origin server 之间终止一段连接并用自己的 cache 回答；conditional GET 通过 validator（如 ETag/Last-Modified）让 origin 判断 304 Not Modified，减少 representation 重传。缓存命中、连接复用、multiplexing 是三个不同的优化点。

### 13 邮件与 FTP 核验层：状态轨迹必须拆开

邮件是 store-and-forward，而不是 sender 到 recipient user agent 的单条实时连接：

UA submission → SMTP relay hop(s) → recipient mailbox → POP3/IMAP access.

SMTP 常用 TCP 25 做服务器间 relay（提交服务的端口与题设可能另给），采用 push；典型命令轨迹包含 220 greeting、HELO/EHLO、MAIL FROM、一个或多个 RCPT TO、DATA、单独一行句点结束、QUIT/221。SMTP reply 表示下一 hop 接受/拒绝，不表示收件人已经阅读。

邮件 content 与 envelope 分离：‘To:’、‘Cc:’、‘Subject:’ 属于内容 header，‘MAIL FROM/RCPT TO’ 是 SMTP envelope 命令；Bcc 可只存在 envelope 而不出现在可见 header。MIME 是内容扩充，不是独立传输协议：它用 Content-Type/Transfer-Encoding 等字段把非 ASCII 或附件编码成 SMTP 可承载的形式；ESMTP 是对 SMTP 命令/能力的扩展，二者不能混写。

POP3 是较简单的 pull/download 模型，常用 TCP 110；可下载并删除或保留副本，但本地文件夹/已读状态同步有限。IMAP 常用 TCP 143，更强调 mailbox 保留在 server、文件夹与多设备状态同步、按需下载部分内容。二者都不是 SMTP 的“接收阶段”同义词。

FTP 的 control/data 分离是应用层设计：control TCP 21 在会话期持续，传命令与认证；每次 LIST/RETR 等目录或文件数据操作另建 data TCP。主动模式由 server 从约定端口 20 连接 client 提供的 endpoint，被动模式由 server 通过 control 告知随机 data port、client 主动连接，后者更适合 NAT/firewall。一次会话若有  $m$  次目录/文件数据传输，典型连接总数为  $1 + m$ ，但同时存在通常只有 control 加当前一条 data。数据完成后 data state 释放，control state 继续保留。

FTP 传“副本”而非像 NFS 那样透明联机访问；ASCII 模式可做换行转换，binary/image 模式逐字节传输。TFTP 的 UDP/停止等待与小型启动用途属于扩展对照，不提升为本册 FTP canonical mechanism。

## 14 408 应用协议流程：消息、状态与停止条件

### 14.1 DNS：递归责任与迭代 referral 分开

stub 向 recursive resolver 发送 query (name、type、class、transaction ID 等); resolver 先查 cache。未命中时, 它依次向 root、TLD 和 authoritative server 发迭代查询, referral 通过 NS 及必要的 glue/address 信息指出下一位 authority; 最终 answer 或 negative result 按 TTL 缓存并返回 stub。收到 response 时必须核对 query identity/匹配字段; TTL 到期后旧 cache 不再直接回答。UDP 是经典普通查询的常见承载, 截断或 zone transfer 等分支可转 TCP, 不能把承载写成唯一语义。

### 14.2 HTTP：请求语义与连接时延分开

一次交互至少推进:

method + target + fields + optional content → server processing → status + fields + optional representation.

性能题先画依赖: DNS 是否命中、transport 是否已有连接、连接能否复用、对象是否串行/并行、内容大小和链路速率。Persistent connection 减少重复建连, cache 减少请求/传输, multiplexing 改变并发关系; 三者不能用同一个“少一个 RTT”口诀替代。

### 14.3 FTP：控制连接决定数据连接怎样建立

client 先建立持久 control connection 并完成认证/命令。需要目录或文件数据时, 主动模式由 server 从约定 data port 连接 client 提供的 endpoint; 被动模式由 server 提供监听 endpoint, client 主动连接。一次 data transfer 完成后数据连接可关闭, control state 继续存在。端口常数和命令细节以题设为准, 稳定边界是 control/data 两条状态轨迹。

### 14.4 Mail：提交、relay 与 access 三段

sender user agent 把 message 提交给 SMTP server; 服务器通过 DNS MX 等信息选择下一跳并用 SMTP relay 推送到 recipient mail server; message 进入 mailbox 后, recipient 再用 POP3/IMAP 等访问协议读取。SMTP reply 驱动提交/转发状态, MIME 只定义可承载的内容表示; 它们都不拥有 mailbox access。停止条件应分别写“已被下一 SMTP hop 接受”和“用户已从 mailbox 访问”, 不能合并成一次端到端 ACK。

#### Owner 边界

DHCP 虽常列在应用层协议表中, 但本项目由 NET04 Own 其配置生命周期; NET-I01 只调用 DNS、TCP 和 HTTP 的输出, 不重复本节报文机制。

## 15 如何选择 transport service

应用选择的不是协议名偏好, 而是约束组合:

| 应用需求                      | 需要观察的 transport 能力      |
|---------------------------|-------------------------|
| 不能丢 byte、允许额外延迟           | reliable ordered stream |
| 保留 message boundary、自定义恢复 | datagram                |



| 应用需求         | 需要观察的 transport 能力               |
|--------------|----------------------------------|
| 极低启动延迟       | connection setup cost            |
| 多路流并发且减少队头阻塞 | multiplexing semantics           |
| 广播 bootstrap | local datagram/broadcast support |
| 安全身份与机密性     | security layer/secure transport  |

“实时应用一定用 UDP”“可靠应用一定用 TCP”都过度简化。应用可在 UDP 之上实现可靠/拥塞控制，或为穿越部署环境选择 TCP；判断必须回到服务需求与实现责任。

## 16 概念边界

| 概念 A                   | ≠ 概念 B                     | 真正区别与题目信号                        | 混淆后果                |
|------------------------|----------------------------|----------------------------------|---------------------|
| Domain name            | ≠ IP address               | 人/组织命名与网络定位；DNS 维护映射             | 把 DNS 当逐跳路由         |
| Domain                 | ≠ Zone                     | 名字树节点/子树与 authority 管理边界         | 授权与层次结构混乱           |
| Recursive query        | ≠ Iterative query          | 被询问方完成解析；被询问方返回 referral         | DNS 报文轨迹画错          |
| Resource               | ≠ Representation           | 被标识对象与某次可传输表示                    | 缓存和内容协商理解错误         |
| HTTP stateless         | ≠ Application has no state | 协议请求独立与业务状态存在可并存                 | 认为登录/cookie 违反 HTTP |
| HTTP semantics         | ≠ HTTP transport version   | method/status/resource 与具体连接承载分离 | 把 HTTP/3 误判为非 HTTP  |
| SMTP                   | ≠ POP3/IMAP                | 邮件传送与 mailbox access             | 邮件全流程协议归属错误         |
| FTP control connection | ≠ FTP data connection      | 命令状态与内容传输分离                      | 端口/连接数量推演错误         |

## 17 做题调用协议

1. 写应用对象：name、resource、file、message 还是 lease/config；
2. 写谁持有 authority/state，谁只是 cache/client；
3. 画 request/response 或 push/pull 的事件方向；
4. DNS 题区分 recursion、iteration、authority、cache TTL；
5. HTTP 题区分 resource、representation、method 和 connection；
6. 邮件题拆成 submission/transfer/storage/access；
7. 性能题画依赖 DAG，再数不可并行的 RTT/transfer；
8. 最后检查 transport 选择是否来自应用约束而非口诀。

## 18 贯穿母例：输入 URL 后第一个 HTTP request 为什么还不能立刻发

```

1 URL identifies scheme + authority + target
2 -> resolver needs server IP (unless cached)
3 -> IP forwarding needs next hop and local MAC (unless cached)
4 -> transport/security context may need establishment
5 -> HTTP request names resource and method
6 -> response carries status + selected representation
7 -> representation may reveal more resource dependencies

```

URL、domain、IP、port、MAC 分别服务不同作用域。把它们压成一句“浏览器访问服务器”，会隐藏几乎所有可诊断的状态交接。

## 19 高频 First Divergence

- 把 DNS 说成从 root 一次返回最终 IP：漏掉 delegation/referral；
- 认为 cache 命中永远是最新真相：忽略 TTL 与弱一致时间窗；
- 写“DNS 用 UDP，HTTP 用 TCP”作为完整机制：把常见承载当唯一语义；
- 把 HTTP stateless 理解成网站不能登录：混淆 protocol context 与 application state；
- 把 response body 等同 resource：混淆对象与 representation；
- 用 SMTP 从 mailbox 下载邮件：没有分开 transfer 与 access。

## 20 一页压缩与复原问题

Name an application object → Choose operation semantics → Represent in messages  
 → Map onto transport → Evolve application state

1. DNS 怎样用 delegation 替代全球单表？
2. cache 为什么同时提高性能并引入旧信息窗口？
3. HTTP resource 与 representation 为什么必须分开？
4. stateless protocol 怎样支撑有状态登录业务？
5. SMTP、POP3/IMAP 和 MIME 分别拥有邮件过程的哪一段？

## 21 来源与校正说明

- 归档笔记《应用层-FTP,SMTP,POP3,DHCP》提供传统应用协议覆盖；《应用层-DNS》《应用层-WWW 与 HTTP》为空文件，未被当作已有正文；
- DNS 的 hierarchy、delegation、resolver、authority 与 cache 边界依据 RFC 1034 校正；
- HTTP 的 stateless semantics、resource/representation 和跨版本语义边界依据 RFC 9110 校正；
- DHCP 已按当前 Ownership 移入 IP Topic，不在应用册重复拥有。